

PORTFÓLIO DE AGENTES INTELIGENTES DE IA

Pesquisa de mercado, benchmarks de frameworks e roteiro técnico para construir 3 projetos de portfólio com padrão de mercado (Sistemas Multiagentes, RAG e API de Automação com Agentes)

Elaborado a partir de fontes oficiais, documentações de frameworks, benchmarks públicos de 2026 e práticas adotadas por empresas e referências do setor (LangChain/LangGraph, CrewAI, NTT DATA, OpenAI, Anthropic, Thoughtworks, entre outras).

Setembro de 2026

Sumário

1. Como este relatório foi construído ...
2. Panorama do mercado: o que empresas e referências constroem com Agentes de IA ...
3. Stack tecnológico recomendado — visão geral comparativa ...
4. Projeto 1 — Sistema Multiagentes de Análise Financeira/Logística ...
5. Projeto 2 — Agente RAG com Otimização de Busca ...
6. Projeto 3 — API de Automação de Agentes com Validação (Back-end) ...
7. Roteiro de execução (Iniciante → Intermediário → Avançado) ...
8. Checklist de diferencial para o README (o que recrutadores avaliam) ...
9. Riscos comuns e como evitá-los ...
10. Fontes e referências consultadas ...

1. Como este relatório foi construído

Este relatório reúne pesquisas em fontes oficiais e de mercado (documentação oficial dos frameworks, comparativos técnicos publicados em 2026, benchmarks públicos de custo/latência, guias de arquitetura de empresas de tecnologia e material de referência como o Technology Radar da Thoughtworks) para responder três perguntas para cada um dos seus 3 projetos de portfólio:

- O que empresas e profissionais de referência realmente constroem hoje nesse nicho de Agentes de IA;
- Qual é o estoque tecnológico (stack) mais usado em produção e por quê;
- Qual é o caminho realista para você, partindo do zero até um nível avançado, sem pular etapas que prejudiquem o aprendizado ou o resultado final no README.

Contexto identificado nos prints enviados

Você é formado em Análise e Desenvolvimento de Sistemas (ADS) e está construindo 3 projetos de portfólio focados em Agentes de IA: (1) Sistema Multiagentes de Análise Financeira/Logística, (2) Agente RAG com base vetorial, e (3) API back-end de automação de agentes com validação. As recomendações abaixo foram construídas para alguém nessa etapa de carreira, com o objetivo de conseguir uma vaga como Desenvolvedor(a) de IA / Engenheiro(a) de Agentes / Backend com IA.

2. Panorama do mercado: o que empresas e referências constroem

2.1 Frameworks de orquestração de agentes — adoção real em 2026

Segundo levantamentos comparativos de 2026 (LangChain, DEV Community, Towards AI e outros), o mercado de frameworks de agentes se consolidou em torno de poucos players dominantes, cada um com um posicionamento claro:

Framework	Popularidade (2026)	Onde é usado em produção	Filosofia
LangGraph	~34–39 milhões de downloads/mês no PyPI (o maior entre frameworks de agentes); versão 1.0 estável desde out/2025	Klarna, Replit, Uber, LinkedIn, Elastic	Grafo de estados explícito (nós e arestas). Mais controle, mais código.
CrewAI	~45 mil+ estrelas no GitHub; ~5,2 milhões de downloads/mês; mais de 100 mil desenvolvedores certificados	+12 milhões de execuções diárias de agentes reportadas pela própria CrewAI	Orquestração por papéis ("equipe" de agentes). Setup rápido, abstração alta.
OpenAI Agents SDK	~10,3 milhões de downloads/mês; ainda em versão 0.x	Ecossistema OpenAI; crescimento acelerado por distribuição nativa	Handoffs explícitos entre agentes especialistas; curva de aprendizado curta.
AutoGen (Microsoft)	Em modo de manutenção desde 2026	Projetos legados; novos projetos migram para o Microsoft Agent Framework	Conversacional entre múltiplos agentes; sucessor unificado com Semantic Kernel.

Leitura prática: existe inclusive um padrão de migração documentado — times prototipam em CrewAI pela velocidade, e quando o fluxo se torna longo, regulado ou precisa de controle fino de estado, migram para LangGraph. A própria CrewAI publicou um guia oficial para esse caminho, e o inverso também acontece. Isso reforça que **dominar os dois é o combo ideal de portfólio**: CrewAI mostra que você entende orquestração baseada em papéis, e LangGraph mostra que você sabe controlar estado, loops e decisões de forma explícita — exatamente o "diferencial de README" que o Projeto 1 pede.

2.2 RAG — o pedido explícito de empresas como a NTT DATA

O enunciado do Projeto 2 já aponta a direção certa: empresas de consultoria e integração de sistemas (como a NTT DATA) contratam especificamente para times que sabem construir busca semântica (RAG) sobre bases de dados privadas — manuais técnicos, contratos, relatórios internos. É um dos casos de uso de IA generativa mais adotados corporativamente em 2026, porque resolve um problema real (documentos internos que ninguém lê) com risco controlado (a resposta é ancorada em documentos, reduzindo alucinação).

O mercado de bancos vetoriais amadureceu bastante em 2026: soluções locais/gratuitas (ChromaDB, FAISS) continuam sendo o padrão para prototipagem e portfólio, enquanto soluções gerenciadas (Pinecone, Qdrant, Weaviate, Milvus) dominam produção em escala.

2.3 Backend de produção para agentes — o que sustenta tudo isso

O Projeto 3 é o que valida, na prática, se você sabe colocar um agente de IA "em produção" de verdade — e não apenas rodando em um notebook. O padrão de mercado observado em repositórios de referência e guias

de arquitetura para 2026 é consistente: **FastAPI** recebe a requisição, devolve uma resposta imediata (202/ID de tarefa), e uma fila assíncrona (**Celery** com **Redis** como broker, ou alternativas mais leves como **ARQ**) processa a cadeia de agentes em segundo plano, persistindo o resultado em **PostgreSQL**.

3. Stack tecnológico recomendado — visão geral comparativa

A tabela abaixo resume, para os três projetos, qual tecnologia usar em cada estágio de maturidade. A lógica é progressiva: você entrega o Projeto 1 (mais simples de orquestração) já dominando o nível "Intermediário", e evolui para o nível "Avançado" no Projeto 3, que é o que mais impressiona em entrevista técnica.

Camada	Nível Inicial (semana 1–2)	Nível Intermediário (padrão do projeto)	Nível Avançado (diferencial)
Orquestração de agentes	CrewAI (Agents + Tasks + Process sequencial)	CrewAI com Process hierárquico ou LangGraph com grafo de estados	LangGraph com checkpointing (Postgres), retomada de falhas e human-in-the-loop
Modelo de linguagem (LLM)	Uma API paga com camada gratuita/baixo custo (ex.: modelo compacto da OpenAI ou Anthropic) via variável de ambiente	Abstração multi-provedor (troca de modelo sem mudar código de negócio)	Roteamento entre modelos por custo/tarefa + fallback automático em caso de erro
Busca vetorial (RAG)	ChromaDB local (arquivo em disco)	ChromaDB/FAISS com reranking e chunking testado	Qdrant ou Pinecone gerenciado, com busca híbrida (vetor + palavra-chave)
Embeddings	Modelo de embedding padrão do provedor de LLM escolhido	Comparação documentada entre 2 modelos de embedding (ex.: OpenAI vs. um modelo aberto)	Modelo open-weight (BGE/E5) avaliado com métricas de recall/precisão (MTEB/BEIR)
Interface	Streamlit (protótipo rápido)	Streamlit com estado de sessão e histórico de conversa	Frontend dedicado (React) consumindo a API via streaming (SSE/WebSocket)
Backend / API	FastAPI com endpoints síncronos	FastAPI + tarefas em segundo plano (BackgroundTasks) ou Celery	FastAPI + Celery + Redis (broker) + PostgreSQL, com padrão de fila distribuída
Saída estruturada / anti-alucinação	Prompt pedindo JSON	Validação com Pydantic + repetição automática em caso de erro de schema	Structured Outputs nativo (tool use / json_schema) com schema estrito no nível do modelo
Observabilidade	Logs simples (print/logging)	Logging estruturado (structlog) + rastreamento de custo por chamada	LangSmith ou OpenTelemetry para tracing de cada etapa do agente
Infraestrutura	Execução local (venv)	Docker + docker-compose (API, worker, Redis, Postgres)	CI/CD simples (GitHub Actions) + deploy em nuvem (Render/Railway/Fly.io) com variáveis seguras

Sistema Multiagentes de Análise Financeira ou Logística

4.1 O que construir (detalhado)

Uma pipeline com 3 agentes especializados, coordenados por um orquestrador:

Agente	Função	Ferramentas típicas
1. Coletor de dados	Faz a varredura de dados de mercado (preços de ações, cotações, fretes, tarifas logísticas) via web scraping ou API pública	requests / BeautifulSoup, ou API (Yahoo Finance, Alpha Vantage, ExchangeRate API)
2. Analista quantitativo	Aplica um modelo estatístico ou fórmula de otimização sobre os dados coletados (ex.: média móvel, desvio padrão, otimização de rota)	pandas, numpy, scipy — chamado como "tool" pelo agente, não como prompt livre
3. Redator de relatório	Consolida os achados dos outros dois agentes em um relatório final, legível para um tomador de decisão	LLM com prompt de síntese + template de saída (Markdown/PDF)

4.2 Stack tecnológica recomendada

- **Linguagem/base:** Python 3.11+
- **Orquestração:** CrewAI para a primeira versão (mais rápido de tirar do papel) — depois reescreva o mesmo fluxo em LangGraph e documente as diferenças no README. Isso, sozinho, já é um diferencial raro em portfólio júnior/pleno.
- **LLM:** API da OpenAI ou Anthropic (comece com um modelo compacto/econômico para controlar custo durante o desenvolvimento)
- **Cálculo/otimização:** pandas + numpy/scipy, expostos como *tools* reais dos agentes (function calling), não como texto gerado pelo LLM — isso evita que o LLM "invente" contas.

4.3 Como resolver o "diferencial no README" pedido

O enunciado pede que você explique três coisas específicas. Aqui está o padrão técnico esperado para cada uma:

a) Arquitetura de comunicação entre os agentes

Desenhe (e inclua no README) um diagrama simples mostrando o fluxo de dados entre os 3 agentes: quem chama quem, o que cada um recebe como entrada e o que devolve como saída. Em CrewAI isso é o encadeamento de *Tasks* (a saída de uma task vira contexto da próxima); em LangGraph, é o *StateGraph* com um estado compartilhado que cada nó lê e atualiza.

b) Como tratar loops de decisão

Documente um limite máximo de iterações (`max_iter`) para evitar que um agente fique tentando indefinidamente. Explique a condição de parada: por exemplo, o agente redator só é chamado depois que o agente analista retorna um resultado validado (não vazio, dentro de um intervalo esperado).

c) Como evitar alucinação entre os agentes

Três táticas usadas em produção, citáveis no README: (1) toda saída numérica vem de uma *tool* determinística (Python), nunca de texto livre do LLM; (2) saída estruturada via Pydantic/JSON Schema entre as etapas, para que um agente não "invente" um campo que o próximo espera; (3) um agente revisor (ou uma validação de código) confere se os números do relatório final batem com os números calculados pelo agente analista antes de liberar a resposta.

Agente RAG com Otimização de Busca

5.1 O que construir (detalhado)

Um assistente que responde perguntas sobre documentos privados (manuais, PDFs de relatórios, códigos de lei) com respostas ancoradas no conteúdo real dos documentos — e não no conhecimento genérico do modelo. A pipeline clássica de RAG tem duas metades:

Etapa	O que acontece	Ferramenta
Ingestão (offline)	PDF/documento é dividido em pedaços (chunks), cada pedaço vira um vetor (embedding) e é salvo no banco vetorial	LangChain (loaders + text splitters) + modelo de embedding
Consulta (online)	A pergunta do usuário também vira vetor, o sistema busca os pedaços mais parecidos por similaridade de cosseno, e o LLM responde usando esses pedaços como contexto	Banco vetorial + LLM (LangChain "retrieval chain")

5.2 Stack tecnológica recomendada

Componente	Escolha recomendada	Por quê
Framework de orquestração RAG	LangChain	É o padrão de facto do mercado para pipelines RAG; maior volume de exemplos, integrações e vagas que citam o nome explicitamente.
Banco vetorial (portfólio)	ChromaDB	Zero configuração de infraestrutura, roda localmente, licença Apache 2.0, ideal para até ~10 milhões de vetores — mais que suficiente para portfólio.
Banco vetorial (mencionar como evolução)	Qdrant ou Pinecone	Qdrant tem o melhor plano gratuito gerenciado e suporte a busca híbrida; Pinecone é o mais citado em vagas por ser "zero-ops" e ter inferência de embeddings embutida.
Modelo de embedding	text-embedding-3-small (OpenAI) para começar; comparar com um modelo aberto (BGE ou E5) da Hugging Face	Permite discutir no README trade-off de custo, latência e privacidade (modelo aberto roda local, sem enviar dados a terceiros).
Interface	Streamlit	Prototipagem de interface em poucas linhas de código, focando o tempo de desenvolvimento na qualidade da busca, não na UI.

5.3 O diferencial pedido: matemática da busca vetorial

O enunciado pede explicitamente para você usar sua base de Matemática para explicar a similaridade de cosseno. Isso é o que deve constar no README:

- **O que é:** a similaridade de cosseno mede o ângulo entre dois vetores (o de embedding da pergunta e o de cada chunk), não a distância absoluta entre eles — por isso é robusta a diferenças de tamanho de texto.

- **Fórmula:** $\cos(\theta) = (A \cdot B) / (\|A\| \times \|B\|)$, onde $A \cdot B$ é o produto escalar entre os vetores e $\|A\|$, $\|B\|$ são suas normas (magnitudes).
- **Por que importa na prática:** quanto mais próximo de 1, mais semanticamente parecido o chunk é da pergunta. Documente no README o valor de *top_k* (quantos chunks você recupera) e o tamanho de chunk/overlap que você testou — isso mostra que você validou a busca, não só copiou um tutorial.
- **Escolha do modelo de embedding:** explique que modelos maiores/mais recentes (ex.: text-embedding-3-large, BGE-large, E5-large) geram clusters semânticos mais "apertados", melhorando recall — ao custo de mais latência e memória. Mostrar essa comparação com números reais (mesmo que num teste pequeno) é o que separa um projeto "copiado" de um projeto "testado".

API de Automação de Agentes com Validação (Back-end)

6.1 O que construir (detalhado)

Este é o projeto que mais pesa para uma vaga de backend/engenharia de software com IA, porque prova que você sabe colocar um modelo em produção dentro de uma arquitetura limpa — e não apenas chamar uma API de LLM num script solto.

6.2 Arquitetura de referência (padrão de mercado 2026)

Camada	Tecnologia	Papel
API síncrona	FastAPI	Recebe a requisição do usuário, valida o payload com Pydantic, enfileira a tarefa e responde imediatamente com um ID de tarefa (padrão 202 Accepted).
Fila / broker	Redis	Transporta as mensagens entre a API e os workers; também pode armazenar o status da tarefa para consulta rápida.
Processamento assíncrono	Celery (ou ARQ para algo mais leve)	Executa a cadeia de agentes de IA em segundo plano, fora do ciclo de requisição/resposta HTTP.
Persistência	PostgreSQL	Guarda o resultado final, o histórico de execuções e o status de cada tarefa (pending/running/done/failed).
Containerização	Docker + docker-compose	Sobe API, worker, Redis e Postgres com um único comando — essencial para o projeto rodar igual em qualquer máquina, inclusive na do recrutador.

6.3 Fluxo de dados típico

- 1. Cliente envia POST /tasks com a tarefa (ex.: "analisar este contrato");
- 2. FastAPI valida o corpo da requisição com um modelo Pydantic e chama task.delay(...);
- 3. FastAPI responde imediatamente com {"task_id": "...", "status": "queued"};
- 4. O worker Celery consome a fila, executa a cadeia de agentes (pode ser o mesmo CrewAI/LangGraph do Projeto 1) e grava o resultado no PostgreSQL;
- 5. Cliente consulta GET /tasks/{id} para saber o status e obter o resultado quando pronto.

6.4 O diferencial pedido: diagrama e robustez

Diagrama de arquitetura

Inclua no README um diagrama (pode ser feito com Mermaid, direto no Markdown do GitHub) mostrando as 4 caixas — API, Redis, Worker, PostgreSQL — e as setas de comunicação entre elas. Isso é o primeiro filtro visual que um recrutador técnico usa para avaliar se você entende arquitetura de sistemas distribuídos, não só "código que funciona".

Respostas não-determinísticas da IA

Documente explicitamente que a saída do LLM é validada com **Pydantic** (ou biblioteca equivalente como *Instructor*) antes de ser salva no banco. Se a IA devolver um JSON fora do schema esperado, o sistema deve tentar novamente automaticamente (padrão "repair loop", até 2–3 tentativas) em vez de quebrar a API. Em 2026, os provedores de LLM oferecem **structured outputs nativos** (json_schema estrito na OpenAI, tool use na Anthropic, responseSchema no Gemini) que garantem conformidade de schema no nível de geração de token — cite qual desses mecanismos você usou e por quê.

7. Roteiro de execução: Iniciante → Intermediário → Avançado

Um roteiro realista para quem está começando agora, pensado para ser cumprido em paralelo com busca de emprego (por isso o Projeto 1 vem primeiro: é o mais rápido de gerar um resultado visível no GitHub).

Semana 1–2 · Nível Iniciante

- Configure o ambiente: Python 3.11+, venv, chave de API (OpenAI ou Anthropic) em variável de ambiente (nunca no código).
- Faça o tutorial oficial do CrewAI (docs.crewai.com) até rodar o exemplo de "pesquisador + escritor".
- Adapte esse exemplo para o Projeto 1: troque os agentes de exemplo pelos 3 agentes descritos (coletor, analista, redator).
- Suba o repositório no GitHub com um README básico (o que é, como rodar, print do resultado).

Semana 3–4 · Nível Intermediário

- Reescreva o mesmo fluxo do Projeto 1 em LangGraph e compare no README as diferenças de controle e código.
- Comece o Projeto 2: monte a ingestão de PDFs com LangChain + ChromaDB, depois a interface em Streamlit.
- Adicione validação de saída com Pydantic nos dois projetos anteriores.
- Escreva testes básicos (pytest) para as funções determinísticas (as "tools" dos agentes).

Semana 5–7 · Nível Avançado

- Construa o Projeto 3: FastAPI + Celery + Redis + PostgreSQL, tudo orquestrado com docker-compose.
- Plugue a cadeia de agentes do Projeto 1 (ou uma nova) como a tarefa assíncrona executada pelo worker.
- Adicione observabilidade básica (logging estruturado) e, se possível, trace com LangSmith ou OpenTelemetry.
- Finalize os 3 READMEs com diagramas (Mermaid), decisões de arquitetura e limitações conhecidas — seção que recrutadores técnicos valorizam muito.

8. Checklist de diferencial para o README

Baseado no que os três enunciados pedem explicitamente e no que recrutadores técnicos de vagas de IA/Backend avaliam em 2026, cada README deve conter:

- **O problema de negócio** que o projeto resolve, em 2–3 frases (não comece explicando código).
- **Diagrama de arquitetura** (Mermaid ou imagem) mostrando os componentes e o fluxo de dados.
- **Decisões técnicas justificadas**: por que esse framework, esse banco vetorial, essa estratégia de chunking — com trade-offs, não só a escolha final.
- **Como rodar o projeto** em 3 comandos ou menos (idealmente via Docker).
- **Limitações conhecidas**: nenhum projeto de portfólio é perfeito, e admitir isso com profissionalismo é mais convincente do que fingir que é.
- **Testes**: mesmo que poucos, mostram maturidade de engenharia.

9. Riscos comuns e como evitá-los

Risco	Como evitar
Gastar toda a cota gratuita de API de LLM em testes	Use um modelo compacto/econômico durante o desenvolvimento e ative logging de custo por chamada desde o início.
Projeto "funciona só na sua máquina"	Use Docker + docker-compose desde o Projeto 3; documente variáveis de ambiente em um arquivo .env.example.
Agentes entrando em loop infinito de decisão	Defina max_iter/timeout em cada agente e um limite global de tempo de execução da tarefa.
LLM "inventando" números em relatórios	Toda operação matemática deve ser uma tool determinística (Python), nunca texto gerado livremente pelo modelo.
Copiar tutorial sem entender	Para cada escolha técnica, escreva no README o "porquê", não só o "como" — é isso que diferencia em entrevista.

10. Fontes e referências consultadas

Pesquisa realizada em setembro de 2026. Links de documentação oficial, comparativos técnicos e benchmarks publicados ao longo de 2026:

Frameworks de agentes

- docs.crewai.com — Documentação oficial do CrewAI (arquitetura Crews & Flows)
- langchain.com/resources/ai-agent-frameworks — LangChain: comparativo oficial de frameworks
- github.com/crewAIInc/crewAI — Repositório oficial CrewAI
- dev.to — "LangGraph vs CrewAI vs OpenAI Agents SDK: Picking Your Agent Framework in 2026"
- pub.towardsai.net — "LangGraph vs CrewAI vs AutoGen: Which AI Agent Framework for Enterprise 2026"
- openagents.org/blog — Comparativo de frameworks open-source com suporte a MCP/A2A

RAG e bancos vetoriais

- firecrawl.dev/blog/best-vector-databases — "Best Vector Databases in 2026"
- datacamp.com/blog/the-top-5-vector-databases — DataCamp: comparativo de bancos vetoriais 2026
- designveloper.com/blog/chroma-vs-faiss-vs-pinecone — Chroma vs FAISS vs Pinecone em 2026
- greennode.ai/blog/best-embedding-models-for-rag — Melhores modelos de embedding para RAG

Backend e produção

- oneuptime.com/blog — "How to Build Background Task Processing in FastAPI"
- github.com/TradeStackDev/fastapi-agent-scaffold — Scaffold de referência FastAPI para agentes
- github.com/briceanu/fastapi-mcp-server — Exemplo de stack FastAPI + LangGraph + Celery + Postgres

Saída estruturada e confiabilidade

- thoughtworks.com/radar/techniques/structured-output-from-llms — Technology Radar da Thoughtworks
- Documentação oficial de Structured Outputs (OpenAI) e Tool Use (Anthropic)

Este relatório foi elaborado por um assistente de IA a partir de pesquisa em fontes públicas disponíveis em setembro de 2026. Frameworks e ferramentas de IA evoluem rapidamente — recomenda-se visitar a documentação oficial de cada tecnologia antes de decisões finais de arquitetura.